

データマイニング特論

第3回

探索的データ分析 (EDA) と可視化

本科目シラバスの確認 3（授業計画）

#	日程	内容
第1回	4/16（木）	ガイダンス、イントロダクション+Python環境構築
第2回	4/23（木）	pandasによるデータ操作の基礎
第3回	4/30（木）	探索的データ分析（EDA）と可視化（オンデマンド）
第4回	5/7（木）	データの前処理と特徴量エンジニアリング
第5回	5/14（木）	分類の基礎 — k近傍法と決定木
第6回	5/21（木）	アンサンブル学習
第7回	5/28（木）	線形分類モデルと回帰
第8回	6/4（木）	モデル評価・選択と不均衡データ
第9回	6/11（木）	クラスタリング手法
第10回	6/18（木）	アソシエーション分析と異常検知
第11回	6/25（木）	テキストマイニング
第12回	7/2（木）	時系列データマイニング
第13回	7/9（木）	深層学習の概観と使いどころ
第14回	7/16（木）	公平性・倫理と再現性
第15回	7/23（木）	まとめ

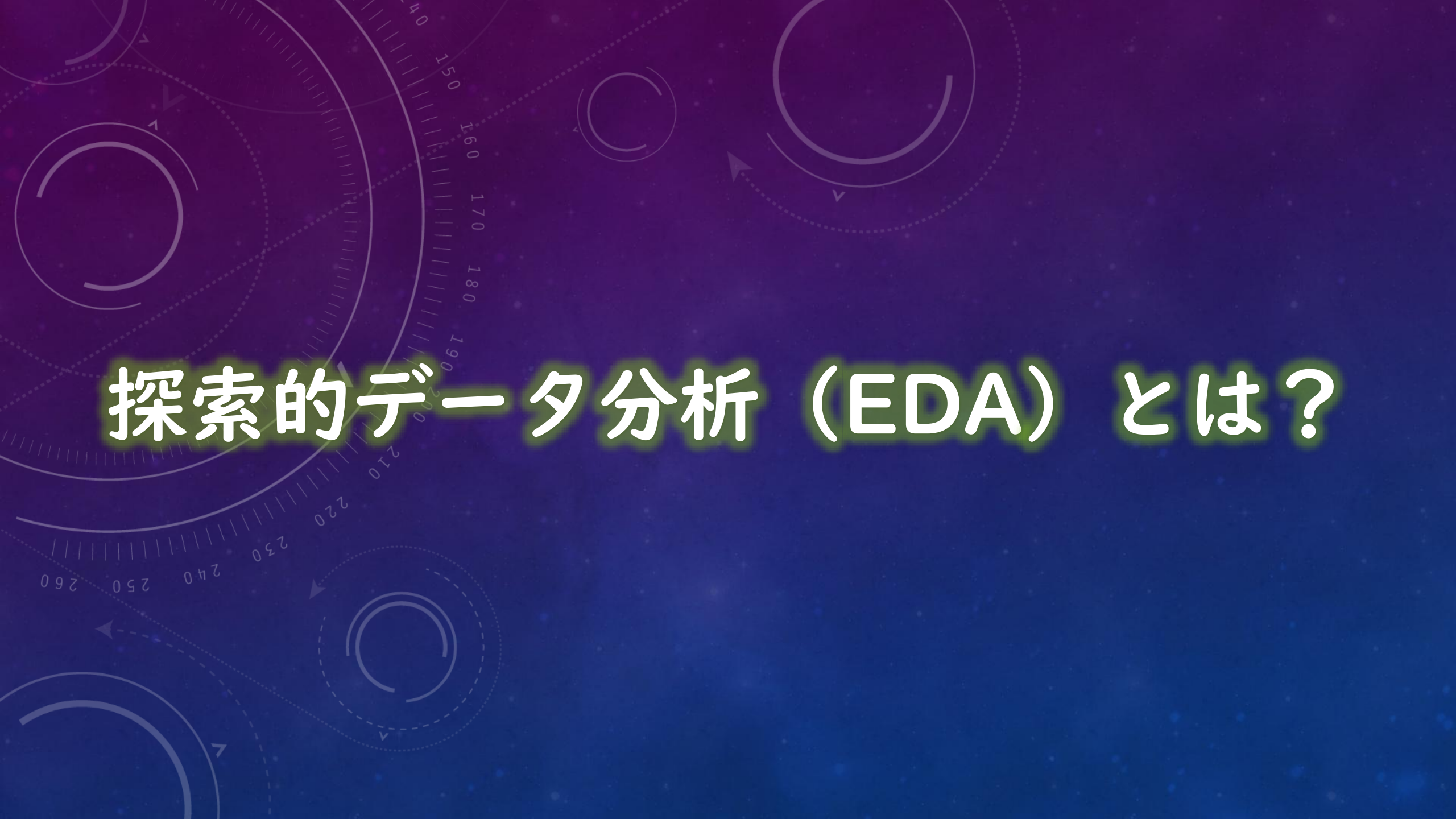
【Point】 やむなく欠席した場合、資料を確認のうえ不明ところは聞いてください。

1

EDAについて

とりあえずデータの中身を確認しよう



The background is a dark blue gradient with a subtle pattern of white dots. Overlaid on this are several faint, light blue circular elements. These include concentric circles, some with dashed lines, and circular arcs with tick marks and degree labels (40, 150, 160, 170, 180, 190, 200, 210, 220, 230, 240, 250, 260). Some of these circles have arrows indicating a clockwise direction.

探索的データ分析 (EDA) とは？

探索的データ分析 (EDA) とは？

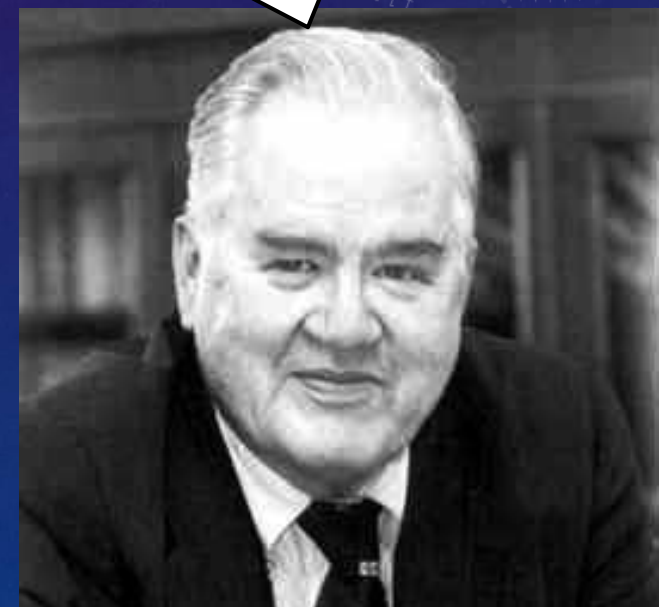
>>> Tukeyの哲学 (1977)

「データが何を語ろうとしているかに耳を傾けること」

⇒探索的データ分析 (Exploratory Data Analysis, EDA) の提唱

	探索的データ分析 (EDA)	確証的データ分析 (CDA)
目的	仮説を見つける	仮説を検証する
手法	可視化・要約統計量	統計的検定・モデル推定
姿勢	柔軟・発見的	厳密・事前計画的
例え	探偵の操作	裁判での立証

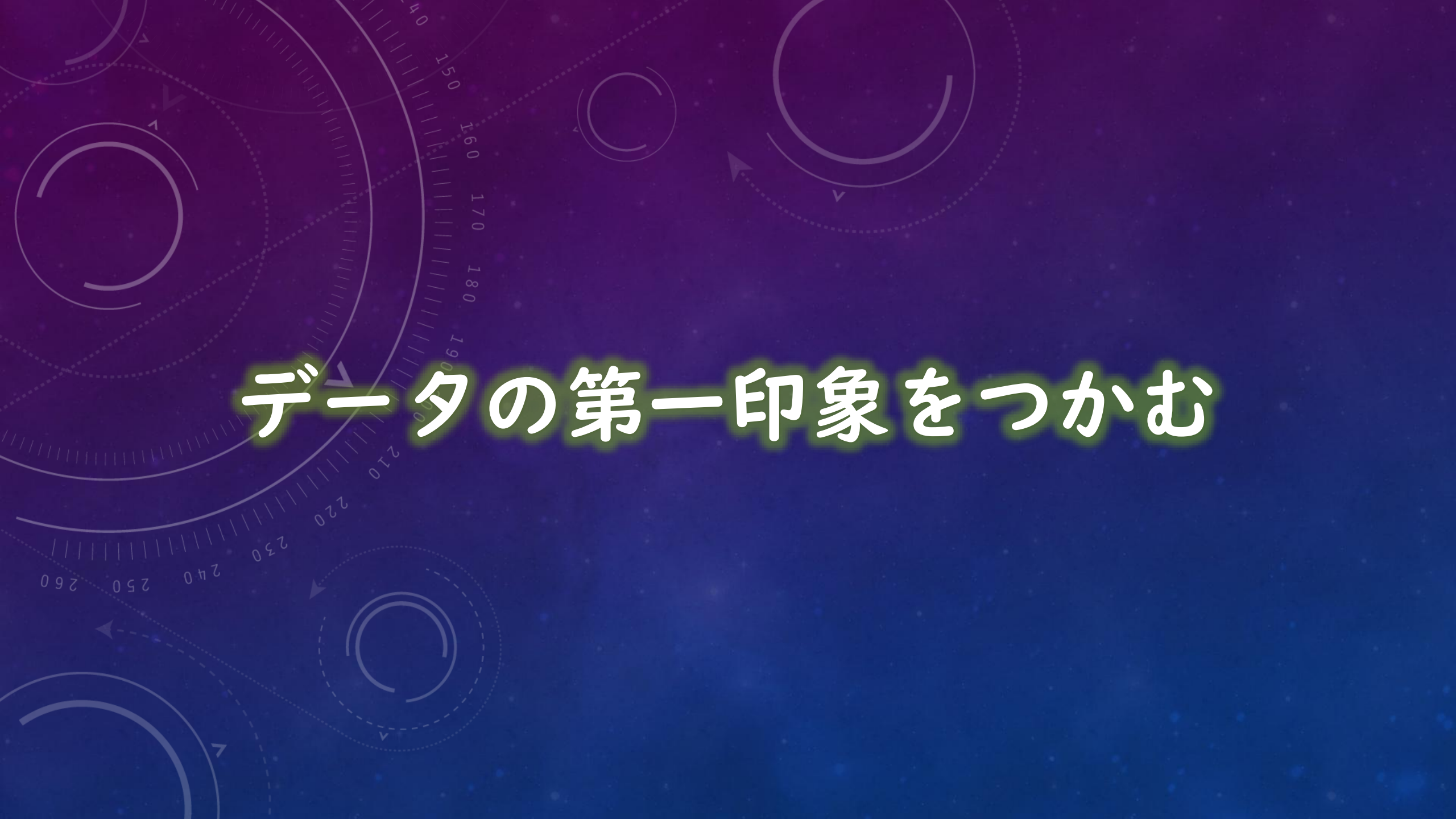
大量データに対して、最適モデルをあれこれ考えてもしょうがねえから、とりあえずデータ全体を俯瞰してみようぜ！！



ジョン・テューキー
(1915～2000)

>>> なぜモデリング前にEDAが必要か

- ・ Garbase In, Garbage Out (GIGO) :
データの品質問題に気付かずモデルを構築すると、結果は無意味になる。
- ・ 特徴量エンジニアリングの着想 :
どの変数が有用か、どんな変換が必要かはEDAから見えてくる
- ・ 前提条件の確認 :
多くの手法は「正規分布に従う」「線形関係がある」等の仮定をもつ

The background is a gradient of dark blue and purple, speckled with small white dots resembling stars. Overlaid on this are several faint, white circular patterns. Some are solid lines, while others are dashed. Some circles have arrows indicating a clockwise direction. A large, semi-circular scale with numerical markings (40, 150, 160, 170, 180, 190, 210, 220, 230, 240, 250, 260) is visible on the left side, resembling a protractor or a circular gauge.

データの第一印象をつかむ

本日の環境準備

いろいろ試してみよう！

```
# 日本語対応のモジュールをインストールする(その1)
```

```
!pip install japanize-matplotlib
```

```
# 日本語対応のモジュールをインストールする(その2)
```

```
!apt-get install -y fonts-ipafont-gothic
```

```
# ライブラリー／モジュールをインポートする
```

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import matplotlib as mpl
```

```
import japanize_matplotlib
```

```
import seaborn as sns
```

```
import warnings
```

```
plt.rcParams['figure.dpi'] = 100
```

```
sns.set_theme(style='whitegrid', palette='colorblind')
```

```
warnings.filterwarnings('ignore')
```

```
# 日本語の設定
```

```
mpl.rcParams['font.family'] = 'IPAPGothic'
```

```
plt.rcParams['font.family'] = 'IPAexGothic'
```

```
# 乱数シード固定(再現性のため)
```

```
np.random.seed(42)
```


データセットの確認

いろいろ試してみよう！

>>> まずはscikit-learnのWineデータセットをつかって基本手順を学ぶ。

3種類のワインの化学成分（13特徴量）と品種ラベルが記録されたデータである。

```
from sklearn.datasets import load_wine

wine = load_wine()
df = pd.DataFrame(wine.data, columns=wine.feature_names)
df['target'] = wine.target
df['target_name'] = df['target'].map({0: 'class_0', 1: 'class_1', 2: 'class_2'})

print(f'データセット概要: {df.shape[0]} 行 × {df.shape[1]} 列')
print(f'¥n目的変数のクラス分布:¥n{df["target_name"].value_counts()}')
```

データセット概要: 178 行 × 15 列

目的変数のクラス分布:

```
target_name
class_1    71
class_0    59
class_2    48
Name: count, dtype: int64
```


基本コマンドの実行 (1)

いろいろ試してみよう！

>>> 以下6つのコマンドは、どんなデータでも最初に行う「定石」手段である。

Step 1: データサイズの確認

```
print(f'行数 × 列数: {df.shape}')
```

行数 × 列数: (178, 15)

Step 2: データ型の確認

```
df.dtypes
```

	0
alcohol	float64
malic_acid	float64
ash	float64
alcalinity_of_ash	float64
magnesium	float64
total_phenols	float64
flavanoids	float64
nonflavanoid_phenols	float64
proanthocyanins	float64
color_intensity	float64
hue	float64
od280/od315_of_diluted_wines	float64
proline	float64
target	int64
target_name	object

dtype: object

基本コマンドの実行 (2)

いろいろ試してみよう！

>>> 以下6つのコマンドは、どんなデータでも最初に行う「定石」手段である。

Step 3: 先頭行の確認
`df.head()`

	alcohol	malic_acid	ash	alcalinity_of_ash	magnesium	total_phenols	flavanoids	nonflavanoid_phenols	proanthocyanins	color_intensity	hue	od28
0	14.23	1.71	2.43	15.6	127.0	2.80	3.06	0.28	2.29	5.64	1.04	
1	13.20	1.78	2.14	11.2	100.0	2.65	2.76	0.26	1.28	4.38	1.05	
2	13.16	2.36	2.67	18.6	101.0	2.80	3.24	0.30	2.81	5.68	1.03	
3	14.37	1.95	2.50	16.8	113.0	3.85	3.49	0.24	2.18	7.80	0.86	
4	13.24	2.59	2.87	21.0	118.0	2.80	2.69	0.39	1.82	4.32	1.04	

Step 4: 基本統計量
`include='all'` でカテゴリ列も表示される
`df.describe().T.round(2)`

	count	mean	std	min	25%	50%	75%	max
alcohol	178.0	13.00	0.81	11.03	12.36	13.05	13.68	14.83
malic_acid	178.0	2.34	1.12	0.74	1.60	1.87	3.08	5.80
ash	178.0	2.37	0.27	1.36	2.21	2.36	2.56	3.23
alcalinity_of_ash	178.0	19.49	3.34	10.60	17.20	19.50	21.50	30.00
magnesium	178.0	99.74	14.28	70.00	88.00	98.00	107.00	162.00
total_phenols	178.0	2.30	0.63	0.98	1.74	2.36	2.80	3.88
flavanoids	178.0	2.03	1.00	0.34	1.20	2.13	2.88	5.08
nonflavanoid_phenols	178.0	0.36	0.12	0.13	0.27	0.34	0.44	0.66
proanthocyanins	178.0	1.59	0.57	0.41	1.25	1.56	1.95	3.58
color_intensity	178.0	5.06	2.32	1.28	3.22	4.69	6.20	13.00
hue	178.0	0.96	0.23	0.48	0.78	0.96	1.12	1.71
od280/od315_of_diluted_wines	178.0	2.61	0.71	1.27	1.94	2.78	3.17	4.00
proline	178.0	746.89	314.91	278.00	500.50	673.50	985.00	1680.00
target	178.0	0.94	0.78	0.00	0.00	1.00	2.00	2.00

基本コマンドの実行 (3)

いろいろ試してみよう！

>>> 以下6つのコマンドは、どんなデータでも最初に行う「定石」手段である。

Step 5: 欠損値の確認

```
missing = df.isnull().sum()
```

```
print('欠損値のある列:', missing[missing > 0].to_dict() if missing.sum() > 0 else 'なし')
```

欠損値のある列: なし

Step 6: ユニーク値数の確認

```
df.nunique()
```

	0
alcohol	126
malic_acid	133
ash	79
alcalinity_of_ash	63
magnesium	53
total_phenols	97
flavanoids	132
nonflavanoid_phenols	39
proanthocyanins	101
color_intensity	132
hue	78
od280/od315_of_diluted_wines	122
proline	121
target	3
target_name	3

dtype: int64

2

データ可視化による探索

実践的な解析のススメ



The background is a dark blue gradient with a subtle pattern of white dots. Overlaid on this are several faint, light blue circular elements. A large circular scale on the left side features degree markings from 140 to 260 in increments of 10. Other smaller circles and arcs are scattered across the frame, some with arrows indicating a clockwise direction.

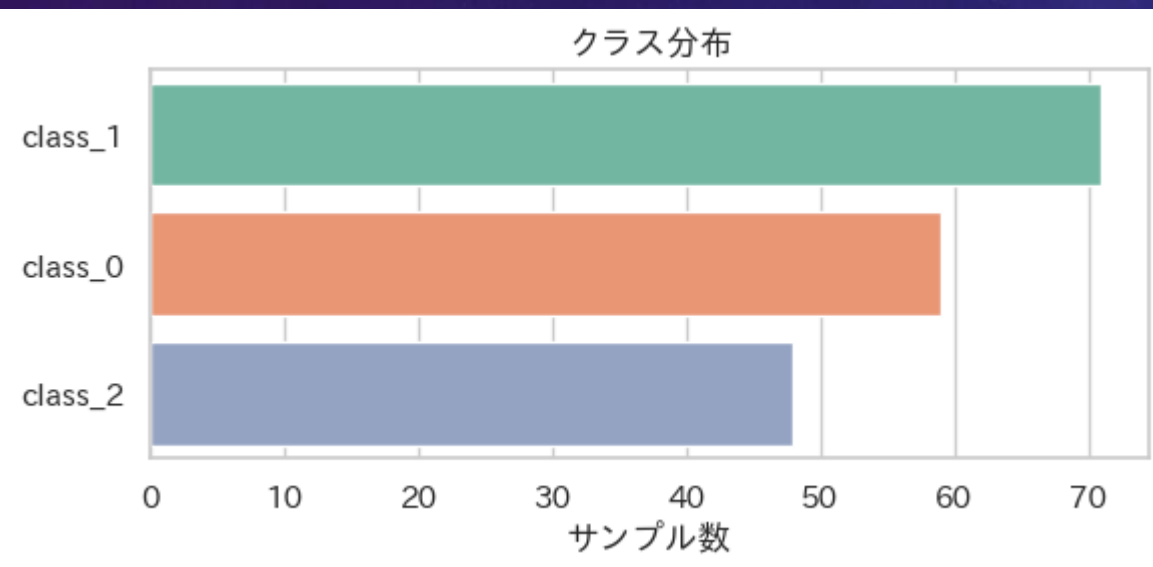
単変量の探索

単変量の探索 (I)

いろいろ試してみよう！

>>> カテゴリ変数の頻度

```
fig, ax = plt.subplots(figsize=(6, 3))
sns.countplot(y='target_name', data=df,
              order=df['target_name'].value_counts().index,
              palette='Set2', ax=ax)
ax.set_title('クラス分布')
ax.set_xlabel('サンプル数')
ax.set_ylabel('')
plt.tight_layout()
plt.show()
```



単変量の探索 (2)

いろいろ試してみよう！

>>> ヒストグラム・・・分布の全体像をつかむ

全数値列のヒストグラムを一覧表示

```
numeric_cols = df.select_dtypes('number').columns.drop(['target'])
```

```
fig, axes = plt.subplots(4, 4, figsize=(16, 12))
```

```
axes = axes.ravel()
```

```
for i, col in enumerate(numeric_cols):
```

```
    sns.histplot(df[col], kde=True, ax=axes[i], color='steelblue')
```

```
    axes[i].set_title(col, fontsize=10)
```

```
    axes[i].set_xlabel('')
```

余ったサブプロットを非表示

```
for j in range(len(numeric_cols), len(axes)):
```

```
    axes[j].set_visible(False)
```

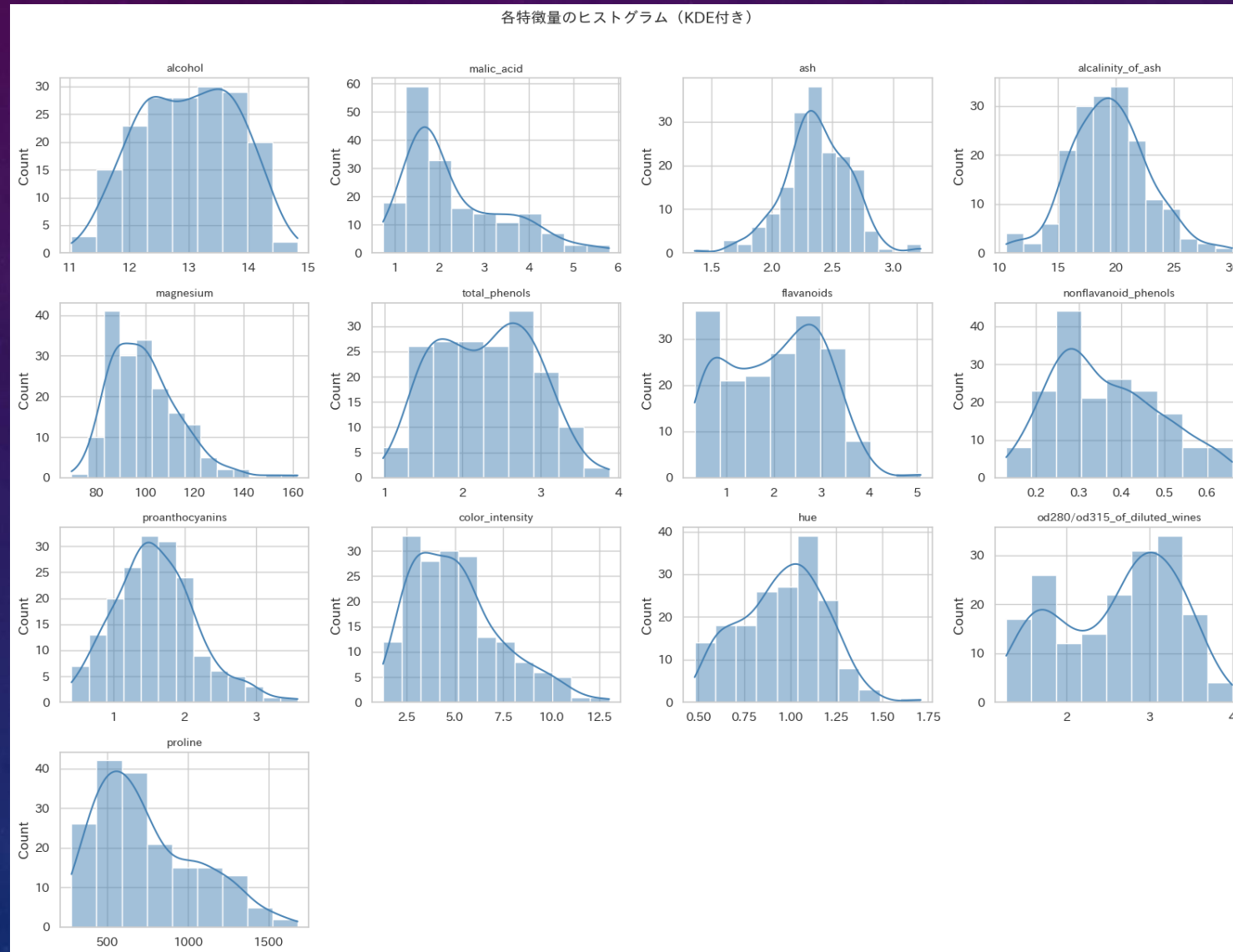
```
plt.suptitle('各特徴量のヒストグラム(KDE付き)', fontsize=14, y=1.01)
```

```
plt.tight_layout()
```

```
plt.show()
```

単変量の探索 (2)

>>> ヒストグラム・・・分布の全体像をつかむ



単変量の探索 (3)

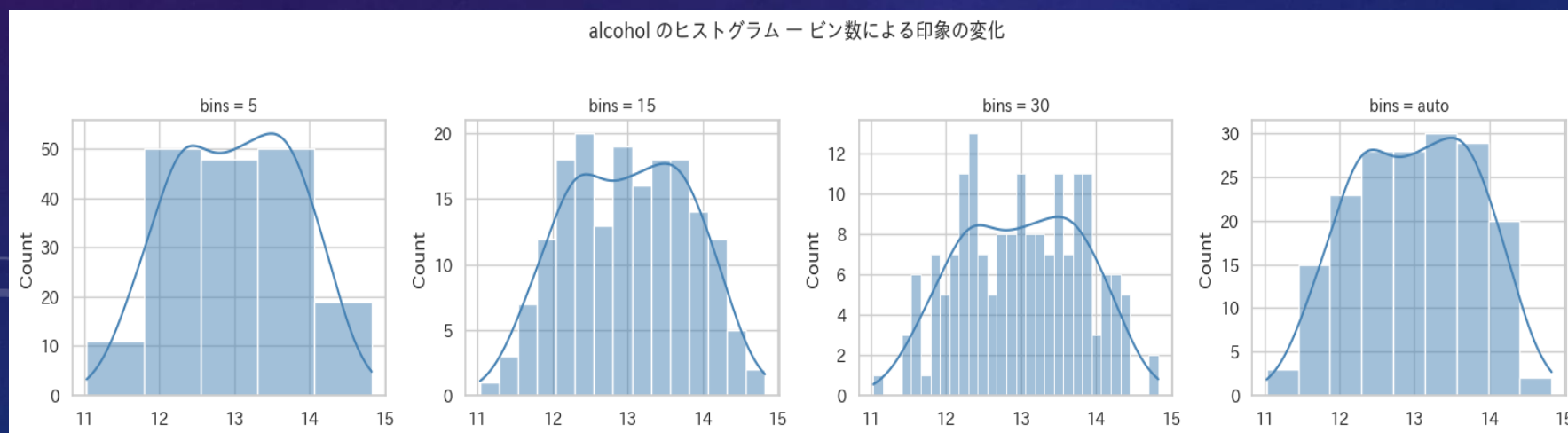
いろいろ試してみよう！

>>> bins数で印象が変わることを体感してみよう。

```
fig, axes = plt.subplots(1, 4, figsize=(16, 3.5))  
bin_options = [5, 15, 30, 'auto']
```

```
for ax, bins in zip(axes, bin_options):  
    sns.histplot(df['alcohol'], bins=bins, kde=True, ax=ax, color='steelblue')  
    ax.set_title(f'bins = {bins}', fontsize=11)  
    ax.set_xlabel('')
```

```
plt.suptitle('alcohol のヒストグラム - ビン数による印象の変化', fontsize=13, y=1.03)  
plt.tight_layout()  
plt.show()
```

同じデータでも、
見え方が変わる？

単変量の探索 (4)

いろいろ試してみよう！

>>> 箱ひげ図 vs. バイオリンプロット vs. スウォームプロット

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

# 箱ひげ図
sns.boxplot(x=df['flavanoids'], ax=axes[0], color='lightblue')
axes[0].set_title('箱ひげ図(Boxplot)')

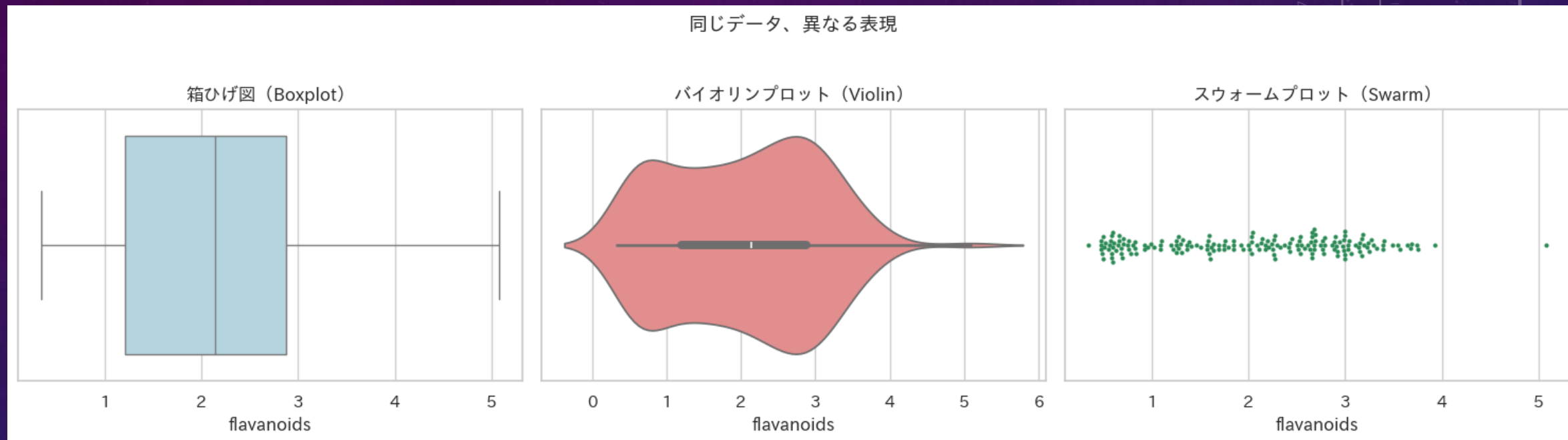
# バイオリンプロット
sns.violinplot(x=df['flavanoids'], ax=axes[1], color='lightcoral')
axes[1].set_title('バイオリンプロット(Violin)')

# スウォームプロット(小規模データ向け)
sns.swarmplot(x=df['flavanoids'], ax=axes[2], color='seagreen', size=3)
axes[2].set_title('スウォームプロット(Swarm)')

plt.suptitle('同じデータ、異なる表現', fontsize=13, y=1.03)
plt.tight_layout()
plt.show()
```

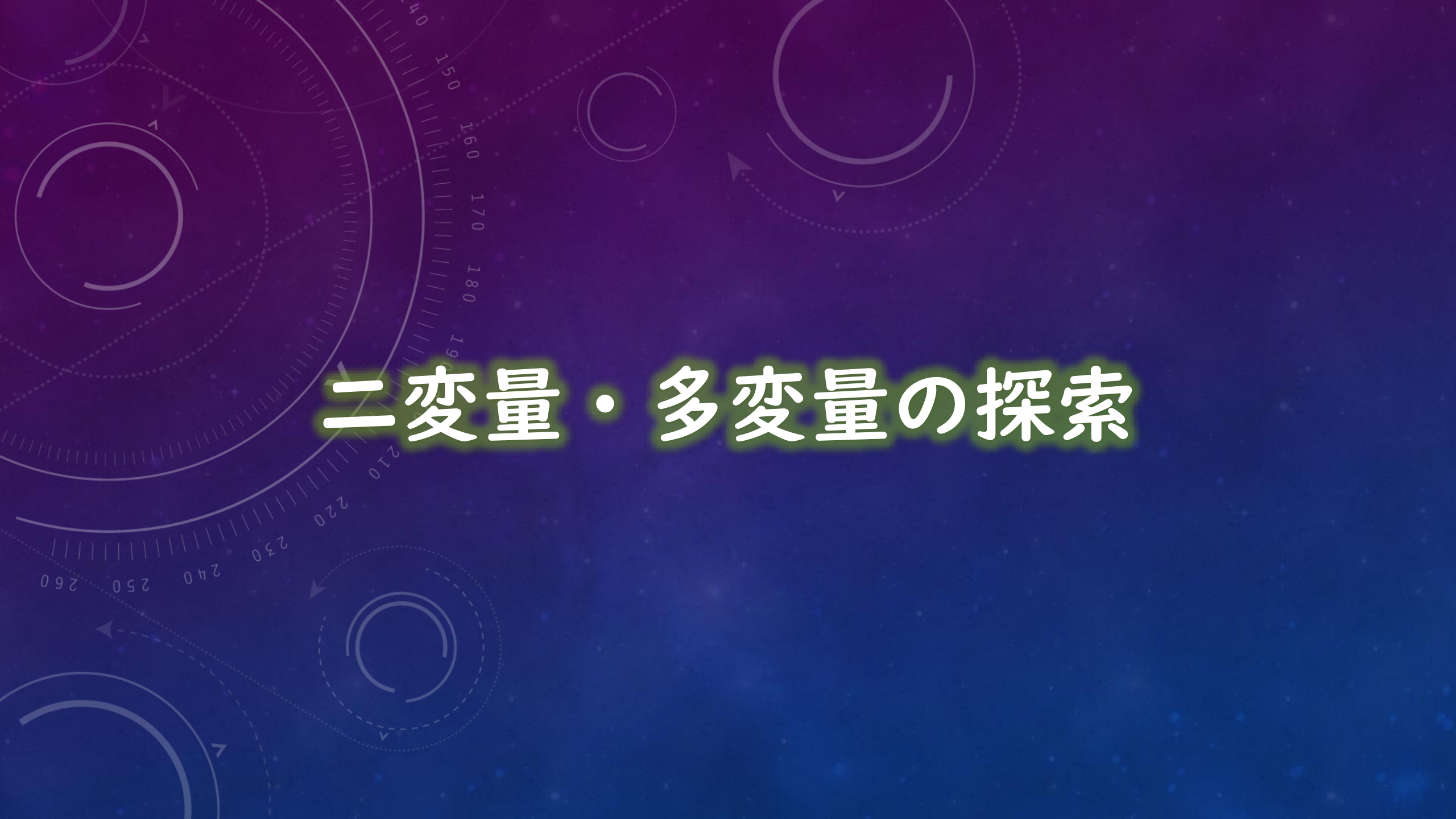
単変量の探索 (4)

>>> 箱ひげ図 vs. バイオリンプロット vs. スウォームプロット



手法	強み	弱み
箱ひげ図	外れ値の特定、四分位数の明治	分布の形状が見えない
バイオリンプロット	分布の形状（双峰性など）が見える	外れ値が目立ちにくい
スウォームプロット	個々のデータ点、N数が直感的にイメージできる	大規模データには不向き

たしかに、表現方法によって伝わる情報が異なる！

The background is a dark blue gradient with faint, light blue geometric patterns. These include several concentric circles of varying sizes, some with dashed lines, and a large circular scale with degree markings ranging from 140 to 260. The text is centered in the middle of the image.

二変量・多変量の探索

二変量・多変量の探索 (I)

いろいろ試してみよう！

>>> 相関行列ヒートマップ

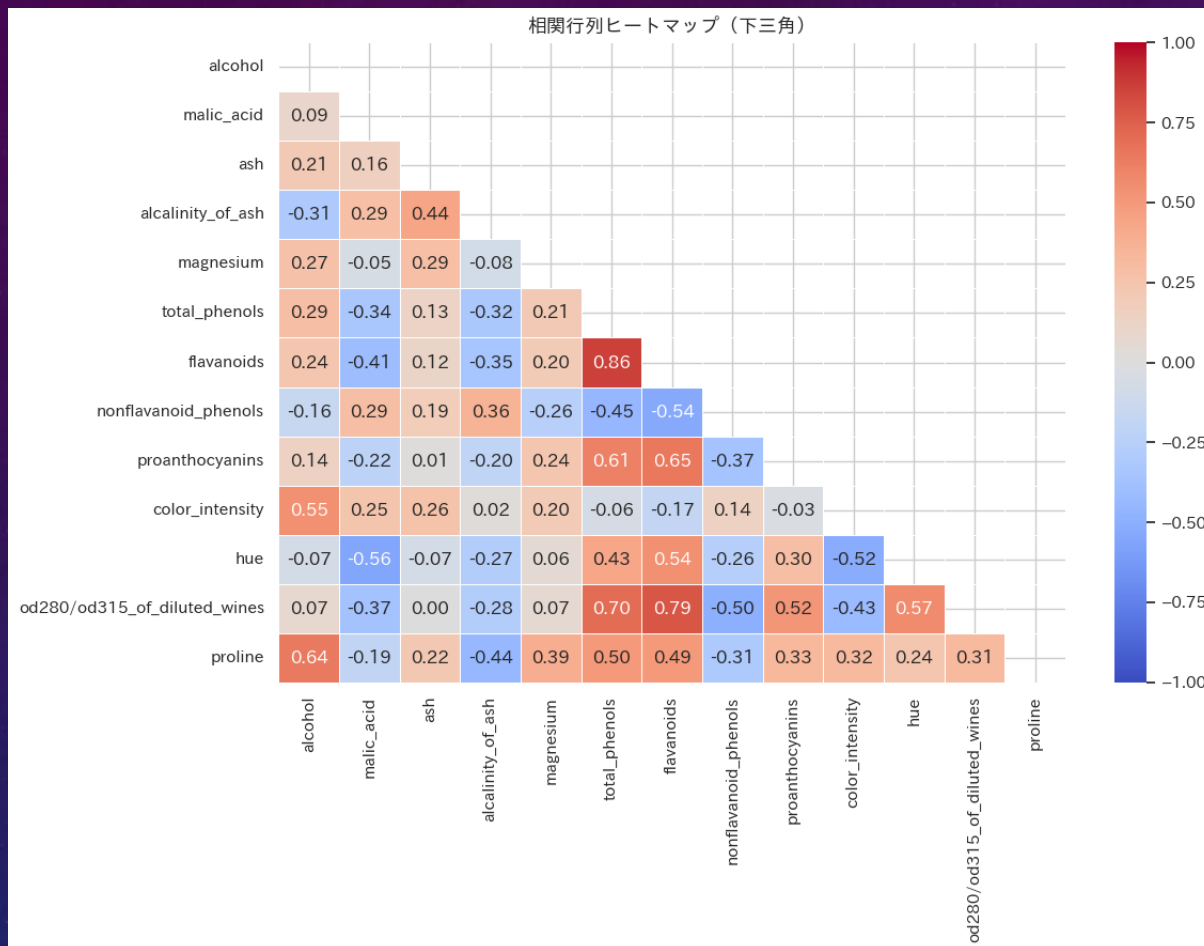
```
corr = df[numeric_cols].corr()

# マスクで上三角を隠す(冗長な情報を排除)
mask = np.triu(np.ones_like(corr, dtype=bool))

fig, ax = plt.subplots(figsize=(12, 9))
sns.heatmap(corr, mask=mask, annot=True, fmt='.2f',
            cmap='coolwarm', center=0, vmin=-1, vmax=1,
            linewidths=0.5, ax=ax)
ax.set_title('相関行列ヒートマップ(下三角)', fontsize=13)
plt.tight_layout()
plt.show()
```

二変量・多変量の探索 (I)

>>> 相関行列ヒートマップ



ポイント

- ・ `mask` で上三角を隠すことで、冗長な情報を排除し見やすくする
- ・ `center=0` で正負の色の対称性を保つ
- ・ `coolwarm` カラーマップは正（赤）と負（青）が直感的に識別できる
- ・ 相関係数は線形関係のみを捉える。非線形の関係は検出できない

二変量・多変量の探索 (2)

いろいろ試してみよう！

>>> Anscombe's Quartet (アンスコム・カルテット、アンスコムの例)・・・相関係数の落とし穴
これは、「必ず可視化すること。数値だけを信用してはいけない」という教訓を伝える古典的な例。
4つのデータセットは、平均・分散・相関係数がほぼ同じだが、散布図で見るとまったく異なるパターンを持っている。

```
anscombe = sns.load_dataset('anscombe')

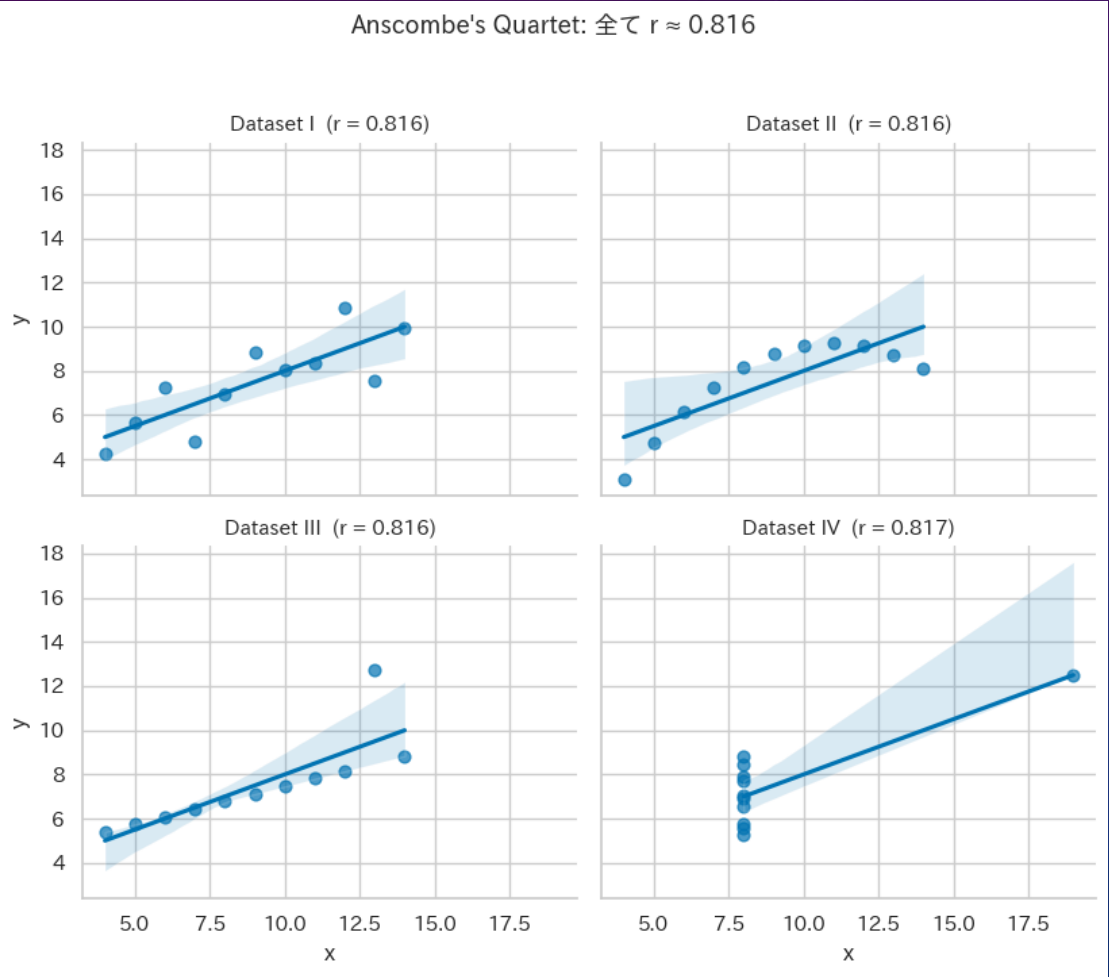
g = sns.lmplot(x='x', y='y', col='dataset', data=anscombe,
               col_wrap=2, height=3.5, aspect=1.2,
               scatter_kws={'s': 40, 'alpha': 0.7})

# 各データセットの相関係数をタイトルに表示
for ax, ds in zip(g.axes.ravel(), ['I', 'II', 'III', 'IV']):
    subset = anscombe[anscombe['dataset'] == ds]
    r = subset['x'].corr(subset['y'])
    ax.set_title(f'Dataset {ds} (r = {r:.3f})', fontsize=11)

plt.suptitle('Anscombe's Quartet: 全て r ≈ 0.816', fontsize=13, y=1.03)
plt.tight_layout()
plt.show()
```

二変量・多変量の探索 (2)

>>> Anscombe's Quartet (アンスコム・カルテット、アンスコムの例)・・・相関係数の落とし穴
これは、「必ず可視化すること。数値だけを信用してはいけない」という教訓を伝える古典的な例。
4つのデータセットは、平均・分散・相関係数がほぼ同じだが、散布図で見るとまったく異なるパターンを持っている。



Dataset	特徴	教訓
I (左上)	正常な線形関係	相関係数が正しく機能するケース
II (右上)	二次関数的な関係	非線形関係を相関係数は捉えられない
III (左下)	1点の外れ値が影響	外れ値が相関を歪める
IV (右下)	1点がすべてを決定	レバレッジポイント (小さな介入で大きな影響を生むこと) の危険性

4つとも $r \approx 0.816$ なのに、データの構造はまったく異なる。必ず可視化することが重要！

二変量・多変量の探索 (3)

いろいろ試してみよう！

>>> ペアプロット・・・目的変数の色分け

列数が多いので、代表的な特徴量を選択

```
selected_features = ['alcohol', 'flavanoids', 'color_intensity',  
                    'od280/od315_of_diluted_wines', 'proline']
```

```
sns.pairplot(df[selected_features + ['target_name']],  
            hue='target_name', diag_kind='kde',  
            plot_kws={'alpha': 0.6, 's': 30},  
            corner=True)
```

```
plt.suptitle('ペアプロット(主要特徴量、corner=True で上三角を省略)',  
            fontsize=14, y=1.02)
```

```
plt.show()
```

二変量・多変量の探索 (3)

>>> ペアプロット・・・目的変数の色分け



ポイント

- ・ 列数が多い場合 (>8列程度) は全列のペアプロットは実行時間がかかるため、事前に列を選択する
- ・ `corner=True` で冗長な上三角を省略できる
- ・ 対角線の `diag\_kind='kde'` で各クラスの分布の重なりが見える

二変量・多変量の探索 (4)

いろいろ試してみよう！

>>> boxplot + stripplot の重ね描き

```
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

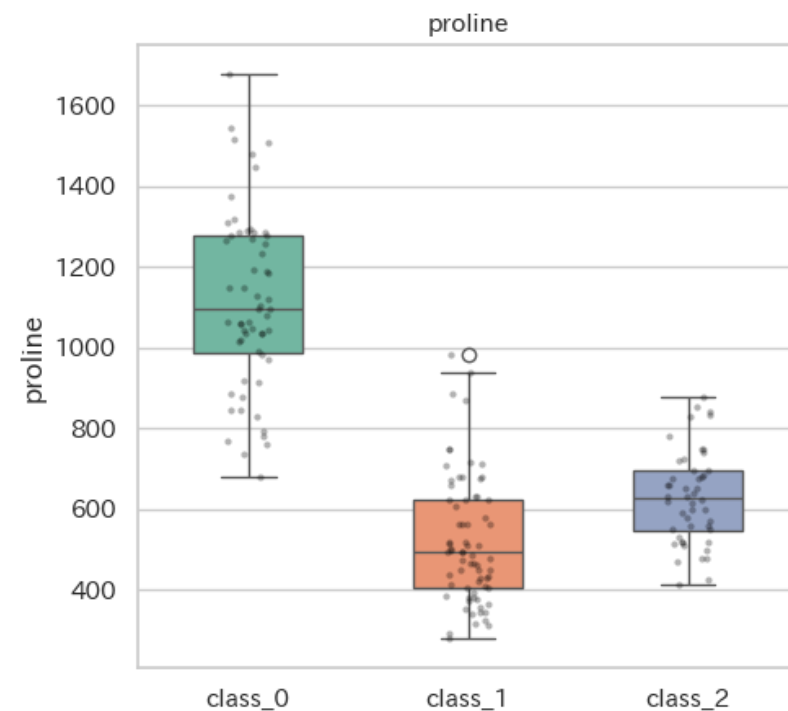
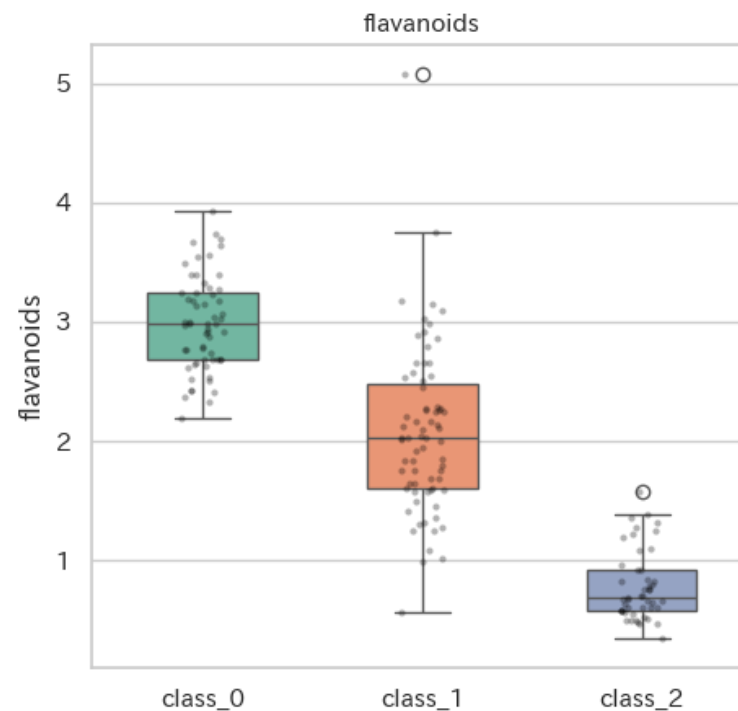
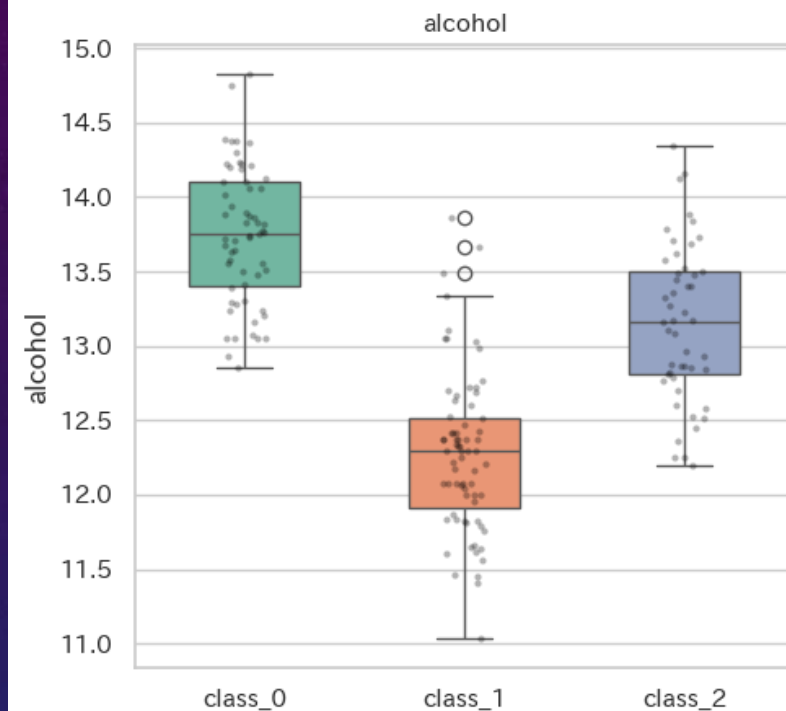
for ax, col in zip(axes, ['alcohol', 'flavanoids', 'proline']):
    sns.boxplot(x='target_name', y=col, data=df, ax=ax,
               palette='Set2', width=0.5)
    sns.stripplot(x='target_name', y=col, data=df, ax=ax,
                 color='black', alpha=0.3, size=3)
    ax.set_xlabel('')
    ax.set_title(col, fontsize=11)

plt.suptitle('クラス別の分布比較(箱ひげ図 + 実データ点)', fontsize=13, y=1.02)
plt.tight_layout()
plt.show()
```

二変量・多変量の探索 (4)

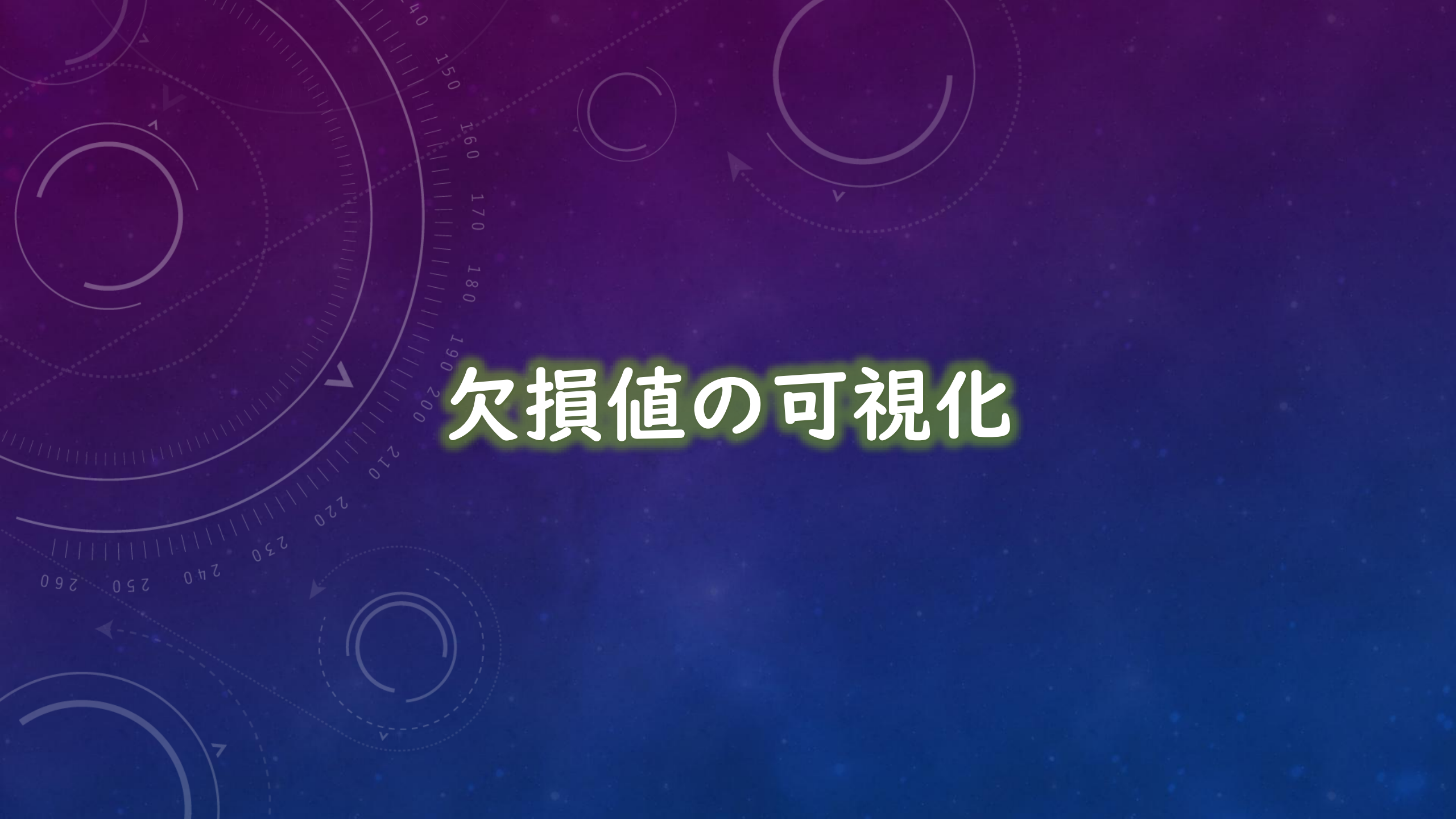
>>> boxplot + stripplot の重ね描き

クラス別の分布比較 (箱ひげ図 + 実データ点)



boxplotだけでは各クラスのN数がわからない。

stripplotを重ねることで、分布の概要とデータ点の密度の両方を1枚の図で伝えることができる。

The background is a dark blue gradient with a subtle pattern of white dots. Overlaid on this are several faint, light blue circular elements. A large circular scale is prominent on the left side, with numerical markings from 140 to 260 in increments of 10. Other smaller circles and arcs are scattered across the frame, some with arrows indicating a clockwise direction. The overall aesthetic is technical and futuristic.

欠損値の可視化

欠損値の可視化 (I)

いろいろ試してみよう！

>>> Wine データには欠損がないため、意図的に3パターンの欠損を導入して
`missingno` の使い方を学ぶ

```
import missingno as msno

# 欠損データの作成(デモ用)
df_missing = df.copy()
n = len(df_missing)

# パターン1: ランダム欠損(MCAR)
df_missing.loc[np.random.choice(n, 20, replace=False), 'alcohol'] = np.nan

# パターン2: 条件付き欠損(MAR) - prolineが低い時にmagnesiumが欠損しやすい
low_proline_idx = df_missing[df_missing['proline'] < df_missing['proline'].median()].index
df_missing.loc[np.random.choice(low_proline_idx, 25, replace=False), 'magnesium'] = np.nan

# パターン3: 共起欠損 - 2列が同時に欠損(同じ測定プロセスの障害を模擬)
shared_idx = np.random.choice(n, 15, replace=False)
df_missing.loc[shared_idx, 'ash'] = np.nan
df_missing.loc[shared_idx, 'alcalinity_of_ash'] = np.nan

print('欠損率 (%) :')
print((df_missing.isnull().sum() / n * 100).round(1).to_string())
```

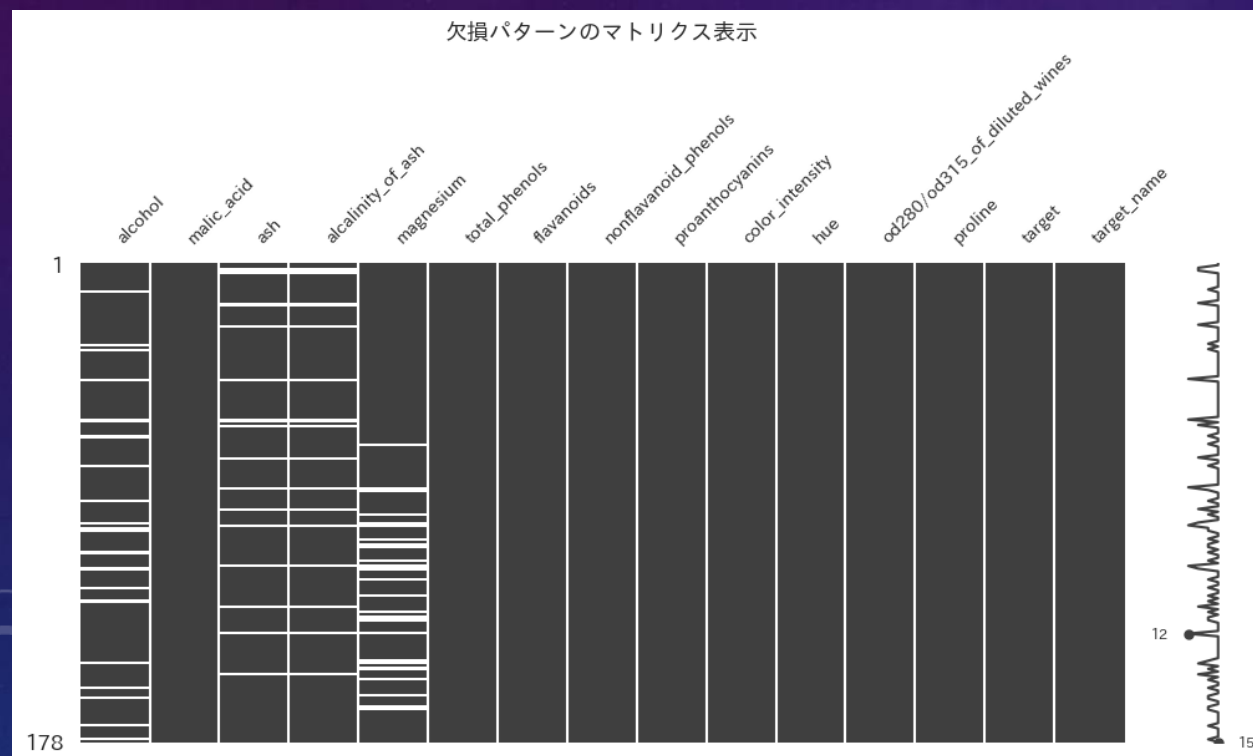
欠損値の可視化 (2)

いろいろ試してみよう！

>>> Wine データには欠損がないため、意図的に3パターンの欠損を導入して
`missingno` の使い方を学ぶ

欠損パターンの全体像

```
msno.matrix(df_missing, figsize=(12, 5), fontsize=10)
plt.title('欠損パターンのマトリクス表示', fontsize=13)
plt.show()
```



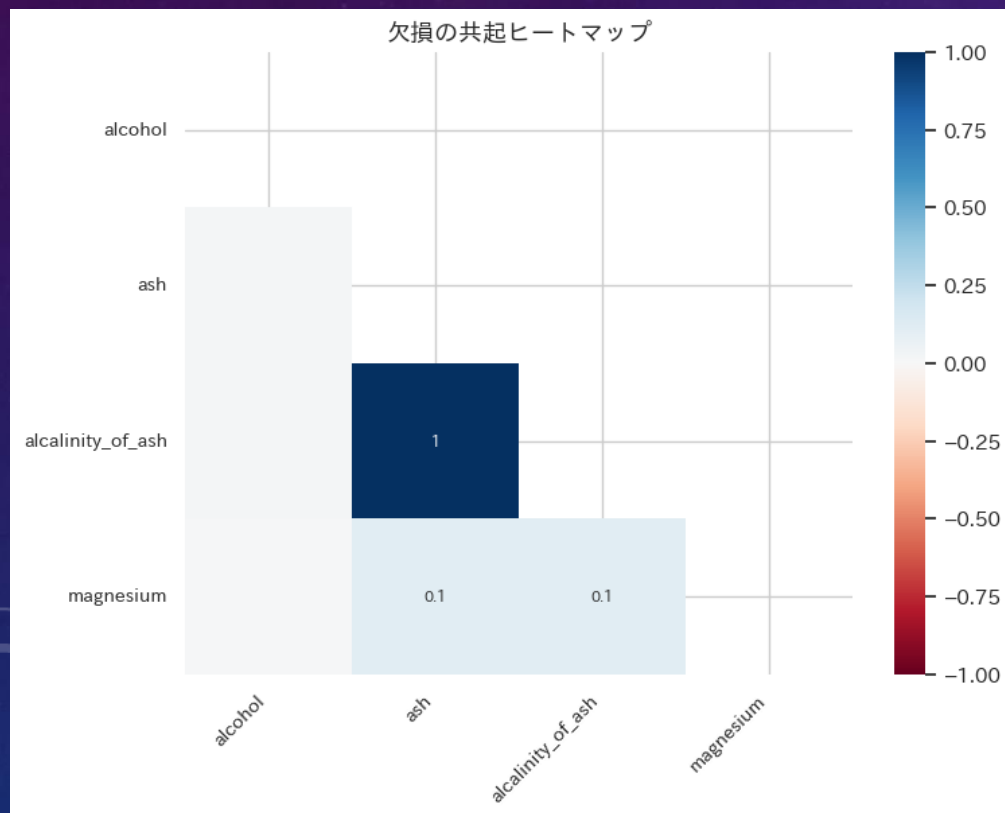
欠損値の可視化 (3)

いろいろ試してみよう！

>>> Wine データには欠損がないため、意図的に3パターンの欠損を導入して
`missingno` の使い方を学ぶ

欠損の共起関係

```
msno.heatmap(df_missing, figsize=(8, 6), fontsize=10)
plt.title('欠損の共起ヒートマップ', fontsize=13)
plt.show()
```



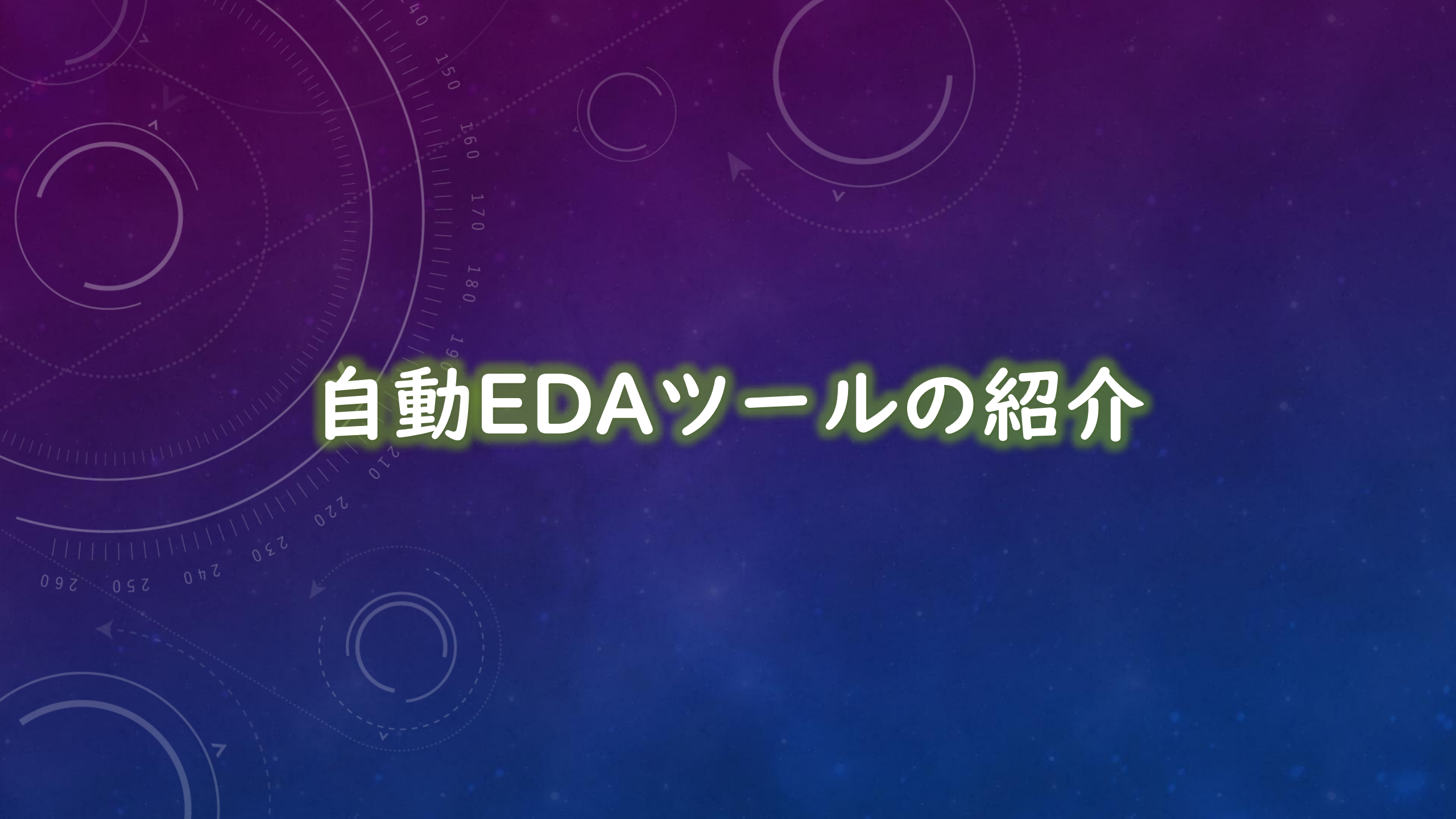
ash と alkalinity_of_ash の間に強い共起関係が見えるはず。
同じ測定プロセスで両方が記録されるため、片方が欠損すれば
もう片方も欠損する状況を模している。

分類	意味	対処法の方向性
MCAR (Missing Completely At Random)	欠損は完全にランダム	リストワイズ削除でもバイアスは小さい
MAR (Missing At Random)	他の観測変数に依存して欠損	多重代入法が有効
MNAR (Missing Not At Random)	欠損値自体に依存	モデルベースの対処が必要

2つの列が同時に欠損している場合

→同じ測定プロセスで記録されている可能性がある。

→データ収集プロセス自体に問題がないか確認すべき！

The background features a gradient from deep purple at the top to dark blue at the bottom, speckled with white dots. Overlaid on this are several faint, light-colored circular elements. A large circular scale with tick marks and numbers (40, 150, 160, 170, 180, 190, 210, 220, 230, 240, 250, 260) is visible on the left side. Other circular patterns include concentric circles and arcs with arrows indicating a clockwise direction.

自動EDAツールの紹介

自動EDAツールの紹介

いろいろ試してみよう！

>>> 自動EDAツールは便利だが、人間にしかできない判断がある。

```
# ydata-profilingのインストール
```

```
!pip install ydata-profiling
```

```
from ydata_profiling import ProfileReport
```

```
# 軽量モードで生成(時間短縮のため minimal=True)
```

```
profile = ProfileReport(df, title='Wine Dataset EDA Report', explorative=True, minimal=True)  
profile.to_notebook_iframe()
```

Overview

Overview Alerts 1 Reproduction			
Dataset statistics		Variable types	
Number of variables	15	Numeric	14
Number of observations	178	Text	1
Missing cells	0		
Missing cells (%)	0.0%		
Total size in memory	29.3 KiB		
Average record size in memory	168.7 B		

自動化は「手間を省く」便利なツールだが、
「判断を代替する」ものではない。

ドメイン知識（専門分野の人による知見や
情報）に基づく問い（「この外れ値は何を
意味するのか？」）は人間にしか立てられ
ない。

3

本日の課題

頑張ってください。



課題

【課題】 期限：2026/5/6（水） 23：59まで

WebClassの『データマイニング特論』アクセスし、
本日の日付が記載される「課題」を期限内に実施してください。

・ WebClass -> データマイニング特論 -> 【第3回（4/30）】課題

（課題1） 本日の演習で「いろいろ試してみよう！」の中で実施した入出力コードを
html形式でエクスポートし、提出しなさい。

html形式にする手順

- ①ipynb形式でダウンロード（ファイル名の例：aaa.ipynb）
- ②再び「ファイル」へアップロード
- ③以下のコマンドを入力

```
!jupyter nbconvert --to html "aaa.ipynb"
```